A typed bookkeeping language for verifiable research with language models:
design, Lean verification, and a case study

Marc Brendecke
Independent researcher, Bochum, Germany

Draft, November 4, 2026

Abstract

When a research project is carried out over months with the help of language models, its written record grows faster than anyone can check it. Three kinds of error then become common and are hard to see in prose: two quantities are compared although they belong to different frames of description, a value that was put in by hand is later cited as a result, and a conditional or local statement is treated as a theorem. We propose QAM, a small typed notation on top of ordinary set theory in which these three steps cannot be written down, together with a simple machine that stores the objects of a record and logs every change. A LaTeX package applies the rules against the three errors while a document is compiled: the author declares each claim with a status, a frame and its premises, and the compiler stops at a promotion, a frame mixing or an input cited as a result. Every claim is linked to the thesis it starts from by a chain of hashes, and the declarations are exported to a file that a checker written in Lean reads. Lean proves that every record this checker accepts has two properties at any distance along its chains of premises: no claim has a higher status than anything it rests on, and no input is lost; a claim declared independent of inputs rests on no claim that carries an input. The notation and the machine are formalized in Lean as well (the machine never leaves a well-formed state, refuses invalid steps without side effects and logs every step). The checks cover only what is declared: a claim declared as proved without premises is accepted as such, the parser of the exported file is not verified, and the hashes are recomputed by a separate script rather than in Lean. We apply QAM to a project in mathematical physics, about a hundred revision rounds and more than sixty pre-registered tests long, and report which errors it caught and which it missed. QAM makes a record checkable in its form; it does not make the content true.

1 Introduction

Language models can now take part in research well beyond answering questions. They write and run numerical code, produce formal proofs, search the literature and draft text, over sessions that add up to months. For a single researcher without institutional support this changes what is feasible. It also changes what can go wrong. A long record written in dialogue with a model accumulates errors of a particular kind, and they are hard to see because each step reads plausibly in prose. In the project described in Section 6 three such classes dominated:

1. *Frame mixing.* A quantity defined in one reference frame, or of one kind (a scale, a readout), is compared with a quantity from another as if they were the same.

2. *Input read as output.* A value that was put in, or chosen to fit, is later cited as if it had been derived.

3. *Promotion.* A local, conditional or numerical statement is restated as a theorem, and later steps build on the stronger form.

Peer review would catch some of these, but a long record produced at the speed of a model is not reviewed step by step, and the model itself is not a reliable reviewer of its own earlier output.

This paper describes a pragmatic answer: a small typed language, QAM, in which the record is kept, and a store machine, MMA, which executes instructions on the record and logs every step. The language does not try to formalize the mathematics or the physics. It types the *objects of the record* — states, layers, operators, readouts, branches, shells, witnesses and tensor composites — and makes the three error classes ill-formed: a comparison across frames needs a declared comparison object, a value that rests on an input carries a witness, and a certificate for a statement can only be formed from a proof. The core of the language and of the machine is formalized in Lean 4 [3].

Contributions.

1. The language QAM over Zermelo–Fraenkel set theory, with its two-sided display for frame-typed quantities, and the instruction layer MML with its transition semantics MMA (Sections 2 and 3).

2. Lean proofs of the properties that make the language usable as a record: disjointness of object kinds, an exactly invertible reindexing, preservation of closure classes along admissible transport, and for the machine preservation, progress for the non-modifying instructions, atomicity, ledger completeness and the absence of false certificates; and a project-independent module that states the three error classes as rules on derivation trees and proves that well-formed trees never promote, never lose or invent an input, and never change frame except through declared maps (Section 4).

3. A LaTeX package, `qam.sty`, that applies the same rules while a document is compiled and stops the compiler at the first violation, that generates the QAM notation of each claim from its declaration, that links every claim to its sources by a chain of hashes starting at the thesis, and that exports the record to a file which a verified Lean checker reads (Section 5).

4. A case study of a research project in mathematical physics, carried out by one author with language models from the beginning, with about a hundred append-only audit rounds and more than sixty pre-registered tests (Section 6).

5. An account of what the discipline caught and what it did not, and of what the formal guarantees mean (Section 8).

The guarantees are about form. QAM can ensure that a record never compares across frames without saying so, never cites an input as a derivation, and never certifies what was not proved. It cannot ensure that a premise is true, that a literature value was read correctly, or that a physical reading makes sense. Section 6 contains an example of each.

2 The language

2.1 Object kinds

QAM adds no axioms to Zermelo–Fraenkel set theory. It fixes eight kinds of record objects as tagged tuples, the first entry being a type tag:

state	$\langle 0, a\rangle$,	layer	$\langle 1, \ell, D\rangle$,
operator	$\langle 2, \ell_1, \ell_2, G\rangle$,	readout	$\langle 3, \ell, W, R\rangle$,
branch	$\langle 4, b, B\rangle$,	shell	$\langle 5, r, s, T\rangle$,
witness	$\langle 6, u\rangle$,	tensor	$\langle 7, p, q\rangle$.

Here D is the carrier of a layer, G the graph of an operator from layer ℓ_1 to layer ℓ_2, W the set of readout values and R the readout rule; a branch records a choice, a shell a refinement step, a witness an external input, and a tensor an admitted composite. Recognition predicates such as $\mathrm{QState}(x) :\Leftrightarrow \exists a\,(x = \langle 0, a\rangle)$ identify the kinds. They are recognition clauses, not an independent axiom system: anything that can be said in QAM can be said in ZF.

2.2 Relations

Three relations carry the content of a record: $\mathrm{QAdmTransport}(o, s, t)$, the operator o carries the state s admissibly to t; $\mathrm{QClEq}(s, t)$, s and t lie in the same closure class; and $\mathrm{QRead}(r, s, w)$, the readout r reads s as the value w. Two schemata relate them: closure equivalence is an equivalence relation, and admissible transport preserves the closure class. A readout r is *closure-compatible* for a map J on values if states in one closure class are read to values with the same image under J. What "admissible" and "closure class" mean is left to the application. The schemata constrain them but do not fix them; both extremes — every transport admissible, or none — satisfy them (Section 4).

2.3 Frames and the two-sided display

Quantities that live in a frame carry it as an index. In the two-sided display ${}^{A}{}_{B}\mathcal{T}^{i}{}_{j}$ the front indices record the frame or readout side and the back indices the structure side. Only an upper index may be contracted with a lower one, which is the ordinary action or readout; two upper or two lower indices may be compared only through an explicitly declared comparison tensor, for example $G_{MT}\,{}^{M}X\,{}^{T}Y$ for a quantity X of frame M and Y of frame T. A comparison across frames without such an object is not a well-formed expression. This is the rule that addresses frame mixing.

2.4 Witnesses, branches and status

A value that rests on an input is written as a readout of a witness, $w = R(u)$ with $u = \langle 6, \dots \rangle$, and a choice made during a construction is a branch. A derivation is therefore visibly different from a fit: the former has no witness in its premises. This is the rule that addresses input read as output. Every statement of a record carries one of the status tags *proved*, *computed*, *conditional*, *reading*, *open* or *withdrawn*, and a statement can only be cited with its tag.

2.5 Reindexing

When the tag namespace is extended, existing tags are shifted by one and the tag 0 is reserved for "no active kind". The shift and its back-translation must be exact on inhabited tags, so that old records remain readable.

3 The store machine

A record is kept in a *store*: a content (objects at addresses and relations between addresses), a guard register (predicates the content must satisfy), a list of snapshots, and a ledger. A store is *admissible* if every relation uses stored addresses and every guard holds.

Instructions (MML). $\textsc{store}(a, x)$ puts an object at an address, $\textsc{link}(a, b)$ adds a relation, SNAPSHOT saves the content, $\textsc{rollback}(k)$ restores the k-th snapshot, SCRUB is a no-op used for cleaning passes, and $\textsc{export}(a)$ releases a stored object.

Transition semantics (MMA). Each instruction has a raw effect, which may be undefined (a rollback to a missing snapshot, an export of a missing address). A step executes the raw effect and checks the guards on the result:

$$\mathrm{exec}(s,i) = \begin{cases} (s \text{ with entry } (i, \textsc{reject}),\ \textsc{reject}) & \text{if the raw effect is undefined,} \\ (s' \text{ with entry } (i, \textsc{cont}),\ \textsc{cont}) & \text{if it gives } s' \text{ and } s' \text{ is admissible,} \\ (s \text{ with entry } (i, \textsc{guard}),\ \textsc{guard}) & \text{otherwise.} \end{cases}$$

A result that violates a guard is never committed; the old content stays and the ledger records the defect. A program is a list of instructions executed in order.

Certificates. The record distinguishes a *candidate*, which the machine can produce for any statement, from a *certificate*, which contains a proof. A candidate becomes a certificate only through a proof supplied from outside the machine. No instruction produces a certificate.

4 What is proved in Lean

The language and the machine are formalized in Lean 4 (version 4.35.0) in four modules of the library `QSFormal` [1]. They use the Lean core only; other modules of the same library, on the physics, use Mathlib [4]. Continuous integration builds the library and rejects `sorry`, `admit` and `native_decide`. Theorem names below are the Lean names.

4.1 Object kinds and reindexing

The eight kinds are the constructors of an inductive type `Obj`; the ZF tag is a function `Obj.tag`.

Theorem 4.1 (`qState_iff_tag`, `not_state_and_op`, `tag_lt_eight`)**.** *An object is a state exactly when its tag is* 0*; no object is both a state and an operator; every tag is less than eight.*

Theorem 4.2 (`back_shift`, `shift_back`, `back_toShifted`, `toShifted_back`)**.** *The shift* $k \mapsto k+1$ *and the back-translation* $a \mapsto a-1$ *on inhabited tags* $a \geq 1$ *are mutually inverse, and the shifted namespace with reserved* 0 *is in bijection with "no kind or a kind".*

4.2 Transport and readout

A *model* of the transport layer is a structure with relations `admTransport`, `clEq`, `read` and the two schemata as fields. Every theorem holds for every model.

Theorem 4.3 (`first_transport_principle`, `chain_clEq`, `chain_readout`)**.** *An admissible transport keeps a state in its closure class, and a closure-compatible readout gives the same value, up to* J*, before and after. The same holds for every finite chain of admissible transports.*

Theorem 4.4 (`totalModel`, `emptyModel`, `empty_chain`)**.** *The schemata have two trivial models: in one every transport is admissible and there is one closure class, in the other no transport is admissible and the classes are singletons. In the second, a chain of admissible transports is trivial.*

The last theorem is a warning as much as a result. The schemata are consistent, but they do not determine what admissible means; all content enters through the choice of the model.

Routes. For frame-typed records there is one model per route $a \to b$, with a composition law as an explicit assumption: an admissible transport from a to b followed by one from b to c is admissible from a to c.

Theorem 4.5 (`path_admissible`, `path_clEq`, `loop_clEq`)**.** *A path through any number of frames is admissible on the direct route and stays in a closure class of it; in particular a loop* $a \to \cdots \to a$ *never changes the class of the route* $a \to a$*.*

4.3 The store machine

Theorem 4.6 (Preservation: `exec_preserves`, `run_preserves`, `exec_cont_admissible`)**.** *From an admissible store no instruction, and no program, leads to an inadmissible one; a step with status* CONT *ends in an admissible store.*

Theorem 4.7 (No stuck states and exact status: `exec_ne_halt`, `exec_cont_iff`, `exec_reject_iff`)**.** *No step ends with status* HALT*. A step has status* CONT *exactly when its raw effect is defined and admissible, and status* REJECT *exactly when its raw effect is undefined.*

Theorem 4.8 (Atomicity: `exec_atomic`)**.** *A step that does not end with* CONT *leaves content, snapshots and guards unchanged.*

Theorem 4.9 (Progress: `snapshot_progress`, `scrub_progress`, `export_progress`)**.** *In an admissible store,* SNAPSHOT *and* SCRUB *always succeed, and* EXPORT(a) *succeeds for every stored address a.*

Theorem 4.10 (Lineage: `exec_logs`, `exec_ledger_length`, `run_ledger_length`)**.** *Every step puts an entry with its instruction and status at the head of the ledger, and a program adds exactly as many entries as it has instructions.*

Theorem 4.11 (Rollback and certificates: `rollback_restores`, `no_false_certificate`)**.** *A rollback to an existing snapshot whose content is admissible restores it exactly. There is no certificate for* `False`*, whatever instructions were run.*

The statements follow the standard shape of type soundness [5, 6] — preservation and progress — adapted to a machine whose steps may be refused. Progress is proved for the three instructions that do not change an admissible content; for STORE, LINK and ROLLBACK it depends on the guards by design, and the exact status characterization says what happens instead. The certificate rule is a small instance of the idea behind proof-carrying code [7]: what is certified carries its proof.

4.4 Status, inputs and frames

The three error classes of Section 1 are stated directly in a module `QAM.Provenance` of a separate Lean package `qam/lean`, which uses the Lean core only (no Mathlib) and does not depend on the rest of the library, or on any physics. A *claim* has a status, a frame and a list of inputs, the identifiers of the values it rests on. Statuses are ordered, open $<$ reading $<$ conditional $<$ computed $<$ proved. Claims are combined by derivation trees with three kinds of node: a *step* derives a statement from one premise, a *join* from two premises, and a *conversion* moves a statement into another frame through a map g. A tree is well formed relative to a set D of declared maps when no node has a higher status than its premises, a join combines premises of one frame only, and every conversion uses a map in D with the right source frame.

Theorem 4.12 (No promotion: `no_promotion`, `proved_only_from_proved`)**.** *In a well-formed tree the conclusion has at most the status of every premise. In particular a conclusion with status* proved *rests only on proved claims.*

Theorem 4.13 (Inputs: `inputs_tracked`, `inputs_sourced`, `independent_iff`)**.** *The inputs of a conclusion are exactly the inputs of its premises: none is lost and none is invented. A conclusion is independent of an input exactly when no premise uses it.*

Theorem 4.14 (Frames: `frames_declared`, `no_maps_same_frame`)**.** *In a well-formed tree every premise reaches the frame of the conclusion through a chain of declared maps. Without declared maps, all premises live in the frame of the conclusion.*

These theorems are elementary, and they are meant to be: the point is that the rules are precise enough to be checked, by a proof assistant and, as the next section shows, by the TEX compiler.

Simplifications. Addresses are natural numbers, relations carry no pointer conditions, and the instructions that clone or merge stores are not formalized.

5 Checks at compile time

A proof assistant checks rules about an abstract record. The record that people actually read is a LaTeX document. The package `qam.sty` [1] brings the rules of `QAM.Provenance` into that document: claims are declared where they are made, and the compiler stops with an error as soon as a declaration breaks a rule.

Commands. `\qamframe{M}` declares a frame and `\qammap{G}{M}{T}` a comparison map from M to T. `\qaminput{k}{M}{s}` declares a value put in by hand, with frame M and status s; its only input is itself. `\qamclaim{k}{M}{s}` declares a premise without inputs, for instance a theorem. `\qamderive{k}{s}{p1,p2,...}` derives a claim from premises, `\qamconvert{k}{s}{G}{p}` moves a premise into another frame, `\qamcompare{a}{b}` records a comparison, and `\qamindependent{k}` asserts that a claim rests on no input. `\qamshow{k}` prints status, frame and inputs. The frame `*` is convention-free: a claim in it, such as a number or a theorem valid in every convention, may be combined and compared with claims of any frame. `\qamcompose{K}{G}{H}` declares the composite of two maps whose frames match.

Checks. `\qamderive` and `\qamconvert` stop when the declared status is higher than that of the weakest premise (promotion); `\qamderive` and `\qamcompare` stop when premises of different frames meet, and `\qamconvert` when the map does not start in the premise's frame (frame mixing); `\qamindependent` stops when the claim rests on an input (input read as result). The package also refuses a key that is used before it is declared or declared twice, so a document must be written in the append-only order of Section 6; because of that order a single compiler pass is enough. An option `mode=warning` turns the errors into warnings.

Example. The frame offset of the case study is computed from two literature values. Declared honestly,

```
\qamframe{M} \qamframe{T} \qammap[G]{G}{M}{T}
\qaminput[\gamma]{gamma}{M}{computed}          % two literature values
\qamclaim[\kappa]{counting}{M}{proved}         % the counting formula
\qamderive[\beta]{beta}{computed}{gamma,counting}
\qamconvert[\beta']{betaT}{conditional}{G}{beta}
```

it compiles, and `\qamshow{beta}` prints [computed | frame M | inputs: gamma]; the conversion gives [conditional | frame T | inputs: gamma]. Both are typeset by the package while this paper is compiled. The two errors of the case study now stop the compiler:

```
\qamderive{beta}{proved}{gamma,counting}
! Package qam Error: Promotion: 'beta' is declared 'proved', but its weakest
(qam)                premise 'gamma' is only 'computed'.

\qamindependent{beta}
! Package qam Error: 'beta' is cited as independent of inputs, but it rests
(qam)                on: gamma.
```

A comparison of `betaT` with a claim of frame M stops with "Frame mixing: ... are compared without a declared map".

Generated notation. The declarations contain everything the QAM notation of Section 2 needs, so the package writes it. The optional first argument of a declaration is the symbol;

`\qam{k}` typesets the claim in QAM notation with its status, `\qam*{k}` without, and `\qamledger` a table of all claims in the order of declaration. For the example, `\qam{gamma}`, `\qam{beta}` and `\qam{betaT}` give

$$^{M}\gamma := R(w_\gamma),\ w_\gamma = \langle 6, \gamma\rangle\ [\mathsf{computed}]$$
$$^{M}\beta \Leftarrow {}^{M}\gamma,\ {}^{M}\kappa,\ W = \{w_\gamma\}\ [\mathsf{computed}]$$
$${}^{T}\beta' = {}^{T}{}_{M}G\,{}^{M}\beta,\ W = \{w_\gamma\}\ [\mathsf{conditional}]$$

— the input as the readout of a witness w_γ, the derived value with its premises and the witness set W it rests on, and the conversion with source and target frame of the map. Nothing here is typed by hand, so the notation cannot drift from the declarations. With the option `export` every claim is also written, one line per claim, to a file `\jobname.qam`, for instance

```
beta := derive(gamma,counting); frame M; status computed; inputs {gamma}
```

together with lines for the maps, comparisons and independence assertions.

Checking the exported record in Lean. The module `QAM.QamFile` reads such a file and checks it line by line against the record before it; the executable `qamcheck` runs this checker on a file. The checker is a Lean function, and Lean proves what its success means. Line by line: every derived claim is fresh, has premises, has at most the status of each of them and shares their frame or is convention-free (`derive_sound`); its listed inputs are exactly those of its premises; a conversion uses a declared map that starts in the premise's frame and ends in the new one (`convert_sound`); an independence assertion holds for a claim without inputs and a comparison only within one frame (`indep_sound`, `compare_sound`). For the whole record: every record the checker accepts, starting from the empty one, satisfies an invariant (`checked_good`) from which the statements about all ancestors of a claim follow, at any distance along its premises — no claim has a higher status than any ancestor, so a proved claim rests only on proved claims (`checked_no_promotion_record`, `checked_proved_record`); every input of an ancestor is an input of the claim, so a claim without inputs has no ancestor with inputs (`checked_inputs_record`, `checked_independent_record`); and no listed input is invented (`checked_inputs_sourced`). The check is independent of the TEX package: a record edited by hand, for instance with an input deleted from a derived claim, compiles nowhere but is still caught by `qamcheck`, and a document compiled with `mode=warning` that contains a promotion produces a record that `qamcheck` rejects. Both cases are part of the continuous integration. The parser that turns text lines into Lean values is not verified; only the checker is.

A hash chain from the starting thesis. A paper usually starts from a thesis, and every later claim is obtained from it by further steps, each of which again has a QAM form. The package records this as a chain of hashes. `\qamthesis{k}{M}{s}{statement}` declares a thesis whose statement is part of the record. Every claim gets a hash, the MD5 sum of its canonical form

kind; *key*; *frame*; *status*; *inputs*; *sources*; *statement*,

where the sources are the hashes of its premises. A derived claim therefore names the exact versions of the claims it was derived from, and a change anywhere upstream — one word of the thesis, a status, an input — changes the hash of every claim below it, while claims that do not depend on the change keep their hashes. `\qamhash{k}` prints a short hash, `\qamledger` adds a hash column and the hash of the whole record, and the export carries hashes, sources and the statement, together with the MD5 sum of the source file. In the example above, computed while this paper is compiled, `gamma` has the hash `C4A2B3DB`, `counting` `8097F8AB`, `beta` `B0541AFD` with these two as sources, and `betaT` `8D193463` with `beta` as source.

The chain is checked twice and independently of TEX: a short script recomputes every hash from the exported fields and the record hash, and the Lean checker verifies the links — a derived claim names exactly the recorded hashes of its premises, in order, and no hash occurs twice (`link_derive_sound`, `link_convert_sound`, `link_leaf_sound`). Lean does not recompute MD5; a statement edited by hand without new hashes is caught by the script, a reworded and rehashed thesis whose dependants still name the old hash is caught by both. For a release the script writes a manifest with SHA-256 sums of source, PDF and record and the record hash, so that a reader of the published files can tell which record was checked. MD5 suffices against accidental change; it does not protect against a deliberate forgery, for which the manifest's SHA-256 sums are the relevant check. Equal hashes mean equal records, not true ones.

What the package does not do. It checks only what is declared. A document that states a result without declaring it is not checked, and a declaration can be wrong: whether `gamma` really is an input is the author's statement, not the package's. In particular a claim declared with `\qamclaim` as proved is accepted without any justification; the package only ensures that nothing derived from it claims more. A fitted value that is later declared again under a new name as a claim without inputs is not recognised as the same value. Frames are names; products of several conventions (signature and normalisation, say) have to be spelled out as frames of their own with declared maps between them. The package guards against slips in a long record; it does not prove its content. It is tested by documents that must compile and documents that must stop with a given error, and the tests run in continuous integration.

6 Case study

6.1 Setting

The project is a programme in mathematical physics that starts from one conjecture, by the author: that matter, light and a tachyon sector are one object read out in three frames [2, 13]. Everything else grew out of it in a dialogue with language models (Claude; ChatGPT and Codex) used from the beginning. By November 2026 the research text had grown to about 1300 pages, of which roughly 85% are protocols, registers and audit rounds. The author is not a physicist; the models wrote most code, the Lean formalization and much of the text.

6.2 Process

- *Append-only rounds.* Results are added as numbered rounds; earlier rounds are never rewritten, and a correction is a new round that names what it corrects. Since the first audited release there have been about a hundred such rounds.
- *Pre-registration.* Every test with a decision criterion is written down before the computation — hypothesis, data, criterion — and committed; the commit time stamps it [9]. More than sixty tests have been registered. Notes that were not blind say so.
- *Scope list and linter.* A fixed list of statements that are not claimed is checked against every round, also mechanically.
- *Recomputation.* Results that carry weight are recomputed with independently written code, at higher precision where needed, and in one series cross-checked between different models. Large rank computations run in parallel on continuous-integration runners.
- *Review.* Before a paper is released, a separate model instance without access to the record reads only the manuscript and writes a referee report.

6.3 What the register shows

Of the registered tests, the register lists more than a dozen as refuted, six as confirmed, and six as invalid by their own criterion — for instance because a required numerical gap was not reached, even though the numbers came out as predicted. Invalid tests are recorded as invalid, not as passed. Four predictions wait for new data. Among the statements of the research text that were withdrawn or corrected are all steps that relied on an external claimed proof of the Riemann hypothesis, an argument for the cosmological constant that is off by a factor of about 10^{121}, and six early predictions, none of which held.

6.4 Error classes and how they were caught

class	example	caught by
frame mixing	A "structural value" of the central parameter equated a readout of one frame with an offset measured in another; a second one labelled two different states with the same symbol	two-sided display: the comparison has no declared comparison tensor
input read as output	The central parameter, put in from two literature values, was cited as derived; a formula reproducing it to 0.018% had an exponent chosen after the fact	witness typing: the value is a readout of a witness
promotion	Local or conditional statements stated as theorems and built upon, e.g. a cosmological-constant chain	status tags and the scope list; audit rounds
external dependence	Proofs that cited a separate paper claiming the Riemann hypothesis	*not* by the language: an audit round traced the citations
numerical artefact	A high-precision result that came from passing 100-digit numbers through a 15-digit text conversion	recomputation at a generic point; withdrawn before it entered a round
literature reading	A counting value read as "corrected" that later literature considers less well founded	independent literature check

6.5 A translation table

For eight key statements of the project the following table gives the standard notation and the QAM form. The QAM form is not shorter; it records what the standard form leaves to the surrounding text.

standard notation	QAM notation	what the QAM form adds
kernel $q = it\,\mathbf{1} + z_k e_k \in \mathbb{C} \otimes \mathbb{H}$	$\ell_n = \langle 1, n, \mathbb{H} \otimes \mathbb{C} \rangle$, $s_q = \langle 0, q \rangle$, $q \in D(\ell_n)$	the layer on which the state lives
compensator $C = \frac{1}{2}(1 + J)$, $C^2 = C$	$o_C = \langle 2, \ell, \ell, G_C \rangle$, $G_C \circ G_C = G_C$	an operator within one layer, not a readout
sector by the sign of $\sigma = z \cdot z$	$r_\sigma = \langle 3, \ell, \{+, 0, -\}, R_\sigma \rangle$	the sector is a readout of the state
exchange of sectors $z \mapsto i\bar{z}$	${}^{T}{}_{M}\mathcal{J}^{i}{}_{j} z^{j} = (i\bar{z})^{i}$; comparison only via G_{MT}	source and target frame; raw comparison ill-formed

standard notation	QAM notation	what the QAM form adds
compensator matrix, $[\mathcal{M}, J] = [\mathcal{M}, C] = 0$	$G_{\mathcal{M}} \circ G_J = G_J \circ G_{\mathcal{M}}$, $G_{\mathcal{M}} \circ G_C = G_C \circ G_{\mathcal{M}}$	nothing beyond the standard form
one generation from $\mathbb{C} \otimes \mathbb{H} \otimes \mathbb{O}$	$\langle 7, \mathbb{C} \otimes \mathbb{H}, \mathbb{O} \rangle$; the choice of J is a branch $\langle 4, b_J, B \rangle$	the construction is visibly conditional
frame offset $\beta \approx 0.549$	${}^{M}\beta = R_\beta(u)$, witness $u = \langle 6, (\gamma_1, \gamma_2) \rangle$	the value rests on an input
matter fraction $\Omega_M = \beta^2$	$\mathrm{QRead}(r_\Omega, s_{\mathrm{now}}, \beta^2)$, layer ℓ_{now}	validity restricted to one epoch

7 Related work

Proof assistants. Lean [3] and its library Mathlib [4] make it possible to verify mathematics, and language models are increasingly used to produce such proofs. QAM is complementary: it does not verify the mathematics of a record, only its bookkeeping, and only a small part of the project's mathematics is formalized.

Type soundness and proof-carrying code. The properties proved for the store machine are the familiar ones of syntactic type soundness [5, 6]. The rule that a certificate carries its proof echoes proof-carrying code [7].

Provenance. The ledger of the store machine is a provenance record in the sense of the W3C PROV model [8], reduced to instructions and outcomes. QAM adds typing of the recorded objects, which provenance models leave open.

Pre-registration. Writing down hypotheses, data and criteria before the analysis is established practice in parts of the empirical sciences, as a response to problems of reproducibility [9, 10]. The case study applies it to computational and theoretical work, where a commit time stamp is enough to fix a test in advance.

Language models in research. Systems such as FunSearch [11] search for mathematical constructions with language models under automatic evaluation, and proposals for automated research pipelines exist [12]. The present setting is different: a single human directs a long, open-ended project, and the problem is not search but keeping an honest record over months. We are not aware of a documented case of comparable length in which the record itself is typed and partly verified; a systematic survey remains to be done.

8 Limits and discussion

What the language does not prevent. QAM secures form, not truth. It did not prevent the dependence on a false external proof; an audit round found it. It does not check whether a literature value is read correctly, whether a premise holds, or whether a physical reading is sensible. Its schemata admit trivial models, so a record can be well-typed and empty. And a well-typed record can still be wrong in every statement that is not about bookkeeping.

Cost. The discipline is expensive. The research text is dominated by protocols, and a reader who wants the results needs a separate, short account. Status tags and witness typing slow down writing. In the case study the cost was worth paying, because the alternative was a long record that nobody, including the author, could trust; whether it is worth paying in other settings is open.

What helped most. In order of effect: pre-registration with time stamps, append-only rounds, recomputation with independent code, the review of manuscripts by a model instance without access to the record, and the typing rules. The typing rules were most useful for the class of error they were designed for, frame mixing, and least useful for errors of premise.

An outside reading. Before release, an independent model instance played a mathematician with four open projects in different fields, searched for existing tools first, then read this paper and tried the package. It judged the frame check and the tracking of fitted against derived values to be useful and not available elsewhere in the document itself, rated QAM as worth trying on one project but not yet for adoption, and found four bugs, which have been fixed. Its main wishes are the next steps below.

Next steps. Leaf claims that must carry a justification — a reference, the name of a Lean theorem or the hash of a certificate, and for computed values the value, script and commit inside the hash; a status order defined by the user, with conditional claims naming their hypotheses; frames with several convention axes; progress for STORE, LINK and ROLLBACK under explicit guard conditions; the clone and merge instructions; a converter that adds `qam.sty` declarations to an existing research text, so that the compiler checks it; and a comparison with electronic lab-notebook and provenance systems.

9 Conclusion

A small typed language over ordinary set theory, a store machine with a complete ledger, and a handful of Lean theorems about them are enough to make three frequent errors of long, model-assisted research ill-formed, and to make every step of a record traceable. They are not enough to make the record true. The case study suggests that the combination with pre-registration, append-only correction and independent review is what makes such a record trustworthy, and that the typing earns its place in that combination as a guard against one specific, common and otherwise hard to see kind of error.

Data availability

The Lean sources, the research text, the audit rounds and the registration notes are in the public repository [1].

A Lean modules and use of language models

Modules. `QSFormal/QAM.lean` (object kinds, reindexing, transport models, chains), `QSFormal/Models.lean` (the two trivial models), `QSFormal/Route.lean` (routed models and paths), `QSFormal/Store.lean` (store, instructions, transition semantics, preservation, rollback, certificates) and `QSFormal/StoreProgress.lean` (no stuck states, status characterization, atomicity, progress, ledger completeness), built with Mathlib in the directory `formal/`. The project-independent part is a separate package in `qam/lean/` that uses the Lean core only: `QAM/Provenance.lean` (derivation trees: no promotion, inputs, frames) and `QAM/QamFile.lean` (the checker for exported records, its line-by-line

and record-level soundness, the hash links), with the executable `qamcheck`. Both are built with `lake build`; the toolchain is fixed in `lean-toolchain`.

Use of language models. The starting conjecture of the case study is the author's. The language, the machine, the Lean formalization and much of this text were developed in a dialogue with large language models (Claude, Anthropic; ChatGPT and Codex, OpenAI). Some ideas came from the models, others from the author; the author chose the questions and decided what to keep. Their output was accepted only through the checks described in Section 6. The author takes full responsibility for the content.

References

[1] M. Brendecke, QuantumSphere-Unfold, `https://github.com/Moriarty1982/QuantumSphere-Unfold`.

[2] M. Brendecke, *Quantum Sphaera Companion: A Structure-First Mathematical Unfolding*, Zenodo, concept DOI 10.5281/zenodo.19210728.

[3] L. de Moura and S. Ullrich, The Lean 4 theorem prover and programming language, in *Automated Deduction — CADE 28*, Lecture Notes in Computer Science **12699**, Springer 2021, 625–635.

[4] The mathlib Community, The Lean mathematical library, in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, ACM 2020, 367–381.

[5] A. K. Wright and M. Felleisen, A syntactic approach to type soundness, *Information and Computation* **115** (1994) 38–94.

[6] B. C. Pierce, *Types and Programming Languages*, MIT Press 2002.

[7] G. C. Necula, Proof-carrying code, in *Proceedings of the 24th ACM Symposium on Principles of Programming Languages (POPL 1997)*, ACM 1997, 106–119.

[8] W3C, PROV-DM: The PROV Data Model, W3C Recommendation, 30 April 2013.

[9] B. A. Nosek, C. R. Ebersole, A. C. DeHaven and D. T. Mellor, The preregistration revolution, *Proc. Natl. Acad. Sci. USA* **115** (2018) 2600–2606.

[10] M. Baker, 1,500 scientists lift the lid on reproducibility, *Nature* **533** (2016) 452–454.

[11] B. Romera-Paredes et al., Mathematical discoveries from program search with large language models, *Nature* **625** (2024) 468–475.

[12] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune and D. Ha, The AI Scientist: towards fully automated open-ended scientific discovery, arXiv:2408.06292 (2024).

[13] M. Brendecke, Three readouts of one kernel: duality sectors, time reversal and a compensator-form projector in $\mathbb{C} \otimes \mathbb{H} \otimes \mathbb{O}$, draft (2026), in [1].